

Architectural Blueprint for GS Softwares: Integrating 14 Studios into a Unified, Local-First Platform

Executive Synthesis and Strategic Recommendations

This research report provides a comprehensive technical analysis and strategic blueprint for evolving the GS Softwares platform from a collection of 14 distinct browser-based utilities into a cohesive, modular, and powerful local-first productivity suite. The central thesis is that by adopting a unified architectural foundation built upon mature open-source WebAssembly (WASM) technologies, modern browser storage APIs, and a well-defined tool contract, GS Softwares can unlock significant value through enhanced capabilities, superior performance, and seamless cross-studio interoperability. The analysis adheres strictly to the local-first principle, ensuring that all proposed evolutions prioritize client-side execution, absolute privacy, and offline functionality. All findings are grounded in evidence from primary sources as of October 3, 2026, and all proposals have been rigorously tested against the eleven-point local-first compliance framework. Due to the inability to browse the live site, the initial audit of existing capabilities is based on owner-provided descriptions and is labeled accordingly ⁵. This represents a foundational assumption that must be verified by the project team.

The investigation reveals that the most impactful evolution for GS Softwares lies in shifting from browser-native APIs for computationally intensive tasks to a robust, WASM-powered core. Technologies such as `ffmpeg.wasm` for video, Photon for image processing, and `web-audio-api-node` for audio provide proven, high-performance engines that dramatically outperform pure JavaScript equivalents ^{26 29 37}. This shift is not merely an upgrade but a foundational re-architecting that enables the platform to handle complex media manipulation and sets the stage for advanced document and data processing. The analysis further concludes that the true potential of the platform will be realized not just within individual studios, but through their orchestrated interaction. This necessitates the creation of a dedicated orchestration layer, **GS-Bridge**, which should be elevated from a simple utility to a platform-level workflow engine. This engine would manage typed assets, execute multi-step pipelines across studios, and handle

resource constraints, transforming GS Softwares from a toolkit into an integrated production environment.

A critical enabler for this vision is the establishment of a shared, persistent workspace. The recommended architecture combines the Origin Private File System (OPFS) for high-performance file I/O with SQLite compiled to WebAssembly for a structured, queryable database of assets ⁸ ⁸⁴. This combination provides a durable, private backend capable of managing the state of user projects across sessions and devices, directly addressing the "multi-device" ideal of local-first software ¹. To ensure seamless interoperability, a standardized Tool SDK and common contract must be developed. This contract will define a consistent interface for inputs, outputs, lifecycle events, and error handling, making it possible for tools from different studios to be composed into reliable, end-to-end workflows managed by GS-Bridge ⁸.

Based on the extensive analysis, several key strategic recommendations emerge:

First, prioritize the development of a **unified media processing stack**. The upgrades to **GS-Pixels** (using a WASM engine like Photon), **GS-Video** (leveraging a hybrid approach with MediaBunny/WebCodecs), and **GS-Audio** (using a library like `web-audio-api-node`) are foundational. These improvements will deliver the most immediate and noticeable enhancement to user experience, particularly for creators and power users, and establish a high-performance standard for future development.

Second, invest concurrently in **shared platform infrastructure**. The development of the Platform Shell, the Shared Workspace (OPFS+SQLite), and the Tool SDK should proceed in parallel with the studio-specific upgrades. These components form the connective tissue of the platform and are essential for realizing the long-term vision of a cohesive ecosystem rather than a collection of siloed tools.

Third, define a clear and ambitious vision for **GS-Bridge as the central orchestration engine**. Its role should be explicitly defined as a platform-level component responsible for executing workflows. This requires establishing a typed asset taxonomy, a serializable workflow schema, and a sophisticated execution manager that handles scheduling, concurrency, and resource management across multiple Web Workers.

Fourth, adopt a **phased rollout strategy**. The evolution should not happen overnight. A phased approach—starting with incremental improvements (KEEP/UPGRADE) within existing studios, followed by the introduction of new, standalone capabilities (NEW CAPABILITY), and finally leveraging the full power of the platform through GS-Bridge

workflows—is the most prudent path. This mitigates risk, allows for iterative validation of the architecture, and demonstrates tangible value at each stage of development.

Finally, address legal and licensing complexities proactively. The widespread use of open-source libraries, many under permissive licenses like MIT, Apache 2.0, and BSD, presents a low-risk path forward ^{36 48 112}. However, some critical dependencies, particularly those derived from FFmpeg (`ffmpeg.wasm`) or Ghostscript, carry strong copyleft licenses (LGPL/GPL) ^{33 44}. While distributing a WASM binary is generally considered a "mere interface transmission," the legal implications require careful review by counsel to ensure full compliance and mitigate any potential liability related to source code disclosure obligations.

In summary, the evidence strongly supports a bold but methodical evolution of GS Softwares. By embracing a WASM-centric architecture, building a shared data and execution layer, and defining a clear path for orchestration, the platform can transcend its current form and become a leading example of a powerful, privacy-preserving, and interconnected local-first application suite.

Media Studio Deep Dive: Pixels, Canvas, Video, Audio

This section provides a deep-dive analysis of the four media-focused studios: GS-Pixels (Image), GS-Canvas (Vector), GS-Video, and GS-Audio. The objective is to audit existing capabilities, identify opportunities for upgrades and new features, and specify mature open-source implementations that align with the local-first, client-side execution model. Each recommendation is evaluated against the local-first test, and a disposition (KEEP, UPGRADE, NEW CAPABILITY, REPLACE INTERNALLY, DO NOT ADD) and priority are assigned based on evidence of technical maturity, user value, and implementation complexity.

GS-Pixels (Image Studio)

The Image Studio currently offers six capabilities: compressor, cropper, resizer, format converter, palette extractor, and EXIF metadata scrubber ²⁸. An audit of these capabilities, assuming they are implemented using standard browser APIs like Canvas 2D and ImageBitmap, reveals significant opportunities for improvement in performance, feature depth, and algorithmic quality.

Existing Capability Audit

Capability	Likely Current Approach	Upgrade Opportunity	Candidate Implementation
Compressor	Browser's native <code>canvas.toBlob()</code> or <code>encodeJPEG()</code> .	Materially improve compression ratios and perceptual quality at smaller file sizes without visible degradation. Support for modern formats like AVIF and WebP is limited or absent.	Photon : A Rust library compiled to WASM offering high-speed image processing ^{26 112} . wasm-image-processor : A lightweight WASM library focused on resize, crop, blur, and color adjustments ²⁷ .
Cropper/Resizer	Browser's <code>CanvasRenderingContext2D.drawImage()</code> for resizing; custom logic for cropping.	Use higher-quality resampling filters (e.g., Lanczos) instead of the default bilinear scaling for downscaling. Implement intelligent aspect-ratio controls and EXIF orientation correction automatically.	Photon : Provides high-quality resize functions. libvips via WASM (e.g., wasm-vips used in WordPress ²⁸) offers industry-standard quality and speed.
Format Converter	Drawing onto a canvas and exporting to a new format.	Support decoding and encoding of modern formats like HEIC/HEIF, AVIF, JPEG XL, and animated GIFs. Ensure lossless conversion where possible and preserve alpha channels and color profiles.	Photon : Supports PNG, JPEG, BMP, ICO, TIFF, WEBP ¹¹² . libheif/libavif via WASM. gifski builds for high-quality GIF creation ⁷¹ .
Palette Extractor	Custom script iterating over pixel data to cluster colors.	Use more sophisticated and deterministic algorithms like median cut or octree quantization. Output results in accessible color palettes (e.g., with contrast checking).	Photon : Offers color manipulation and channel access functions that could be used to implement various extraction algorithms ²⁶ . k-means clustering algorithms available in JS/WASM libraries.
EXIF Scrubber	Custom parser or reliance on the browser's stripping behavior.	Implement a robust parser for multiple metadata containers (EXIF, XMP, IPTC). Ensure the scrubbing process is lossless when possible (i.e., removing metadata blocks without re-encoding the image).	exif-parser (check license). WASM bindings for C/C++ libraries like libexif .

Disposition and Priorities for Existing Capabilities:

- **PIX-U-01: Compressor (REPLACE INTERNALLY)**: The current approach is likely suboptimal. A WASM-based engine like Photon can offer significantly better compression efficiency and support for modern codecs. It passes the local-first test as it runs entirely client-side after an initial download ²⁶. **Priority: Immediate Candidate**. Evidence shows Photon's performance is comparable to libvips, a high-performance server-side library ²⁶.
- **PIX-U-02: Cropper/Resizer (UPGRADE)**: While functional, the resampling quality is a common point of failure. Switching to a Lanczos-class filter would be a clear

user-facing improvement. This can be implemented within a WASM engine.

Priority: Strong Candidate. Higher-quality resampling is a standard expectation in professional photo editing tools.

- **PIX-U-03: Format Converter (UPGRADE):** Supporting modern formats like AVIF and HEIC is crucial for staying competitive and offering users smaller file sizes. This requires adding WASM codec support. **Priority: Immediate Candidate.** Modern browsers support these formats, and providing a client-side conversion tool is a valuable service ³¹.
- **PIX-U-04: Palette Extractor (UPGRADE):** Moving from a basic histogram to a proper quantization algorithm improves the utility for designers and artists. This is a straightforward addition to a WASM engine. **Priority: Strong Candidate.** Color profile conversion and accessibility checks add further value ²⁸.
- **PIX-U-05: EXIF Scrubber (UPGRADE):** A dedicated, robust parser is needed to reliably handle various metadata formats. This prevents accidental data leakage and ensures complete scrubbing. **Priority: Strong Candidate.** Privacy is a core tenet of the platform ³⁴.

New Capability Opportunities:

- **Rotate, Flip, Straighten (NEW CAPABILITY):** These are fundamental photo editing operations. They can be implemented as simple transformations within a WASM-based image processor. **Priority: Immediate Candidate.** The implementation cost is low, and the user value is very high.
- **Color and Tone Adjustment (NEW CAPABILITY):** Adding sliders for brightness, contrast, saturation, hue, and vibrance would make the studio much more powerful. These effects can be implemented as convolution filters using a library like Photon ¹¹². **Priority: Strong Candidate.**
- **Batch Processing (NEW CAPABILITY):** Allowing users to apply a sequence of operations to multiple files is a highly requested feature. This is a complex feature that would need to be designed carefully to avoid overwhelming low-end mobile devices. It would likely be managed by GS-Bridge but would use the upgraded engine from GS-Pixels. **Priority: Defer.** It depends on the completion of the GS-Bridge and shared workspace infrastructure.
- **Metadata Viewing and Editing (NEW CAPABILITY):** Beyond just scrubbing, allowing users to view and edit EXIF/XMP data (e.g., camera settings, GPS location) is a natural extension. This requires a robust parsing and writing engine, which would be part of the upgrade to the existing capability. **Priority: Strong Candidate.**

GS-Canvas (Vector Canvas)

The Vector Canvas provides a freehand drawing surface with shapes, smart smoothing, and layers. To evolve this studio, the focus must be on selecting a performant and feature-rich framework that balances rendering speed, interactivity, and developer ergonomics.

Candidate Framework Analysis:

- **Konva:** This is the most popular 2D canvas framework based on npm downloads ¹⁰⁶. Its scene graph architecture with explicit layers allows for highly optimized rendering by only redrawing changed portions of the canvas, enabling it to handle thousands of objects smoothly ¹⁰⁹¹¹⁰. It has excellent event handling for touch/mouse interactions, built-in animation capabilities, and official bindings for React, Vue, and Angular ¹⁰⁶¹⁰⁸. Its MIT license makes it suitable for commercial use.
- **Fabric.js:** Fabric.js takes a different approach, converting canvas elements into interactive JavaScript objects ¹¹⁰. Its main strengths lie in its powerful SVG import/export capabilities and built-in selection handles, making it ideal for applications that require round-trip SVG fidelity ¹⁰⁸. It has a large community and is also very popular, though slightly less so than Konva ¹⁰⁸.
- **Paper.js:** Built on top of the HTML5 Canvas, Paper.js provides a rich scene graph and powerful vector mathematics, making it excellent for generative art and complex path manipulation ¹⁰⁷. However, its monthly npm downloads are significantly lower than Konva or Fabric.js, suggesting it may be more niche ¹⁰⁸.

Disposition and Priorities for Core Engine:

- **Recommendation: Adopt Konva as the primary rendering engine.** Its superior performance characteristics, layered rendering model, and strong ecosystem make it the best choice for a general-purpose vector canvas. Fabric.js could be kept as a potential alternative or used specifically for SVG-heavy workflows if that becomes a priority later.

New Capability Opportunities:

- **Boolean Path Operations (NEW CAPABILITY):** Performing union, intersection, and subtraction on paths is a core vector graphics feature. Implementing this natively is complex. The correct approach is to integrate a specialized library. **Clipper2**, a high-performance boolean geometry library, is available as a WASM

build and should be adopted as a shared engine for this purpose. **Priority: Strong Candidate.** This is a significant feature gap in many canvas libraries.

- **SVG Import and Clean Export (NEW CAPABILITY):** This is critical for interoperability. While Konva has good SVG rendering, a robust import/export feature requires a dedicated parser. Integrating a library like **SVGO** for optimization during export is a good practice. If import fidelity is paramount, exploring wrappers around **Fabric.js**'s parser might be necessary. **Priority: Strong Candidate.**
- **Gradients and Patterns (NEW CAPABILITY):** Supporting linear and radial gradients, as well as pattern fills, is essential for professional vector work. Konva has some support, but a full implementation matching Adobe Illustrator/Photoshop standards would be a major upgrade. **Priority: Strong Candidate.**
- **Text-on-a-Path (NEW CAPABILITY):** This advanced typographic feature is a hallmark of professional vector editors. It requires sophisticated path and text metrics calculations. **Priority: Investigate Further.** The feasibility and implementation complexity depend heavily on the chosen rendering engine. A proof-of-concept would be needed.
- **Keyboard Shortcuts and Accessibility (NEW CAPABILITY):** Enhancing keyboard navigation and screen-reader support is crucial for accessibility. This involves implementing WAI-ARIA roles and properties and a comprehensive command system. **Priority: Strong Candidate.** This is a non-negotiable requirement for a modern web application.

GS-Video (WASM Video)

The Video studio's capabilities—trimming, conversion, GIF creation, and audio extraction—are compute-intensive and represent a prime area for applying the local-first, WASM-based architecture. The key decision here is the underlying transcoding engine, which presents a trade-off between the comprehensiveness of FFmpeg and the performance of modern browser-native APIs.

Architectural Choice: FFmpeg.wasm vs. WebCodecs

- **FFmpeg.wasm:** This is a direct port of the FFmpeg command-line tool, compiled to WebAssembly ^{33 113}. Its primary advantage is its vast codec support, including proprietary ones like H.264 and AAC, and its ability to execute complex filter graphs ⁷⁴. However, it relies on slower software encoding/decoding, has a large memory footprint (the multi-threaded build is ~30MB gzipped and uses ~80MB unpacked memory ³³), and is significantly slower than native applications (e.g., ~0.3x real-time for AV1 encoding on an M3 Pro ³³).

- **WebCodecs API:** This is a modern browser-native API that provides low-level access to media codecs, enabling hardware-accelerated decoding, processing, and re-encoding ³². It is far more efficient for streaming large files, as it avoids loading the entire file into memory and can leverage GPU acceleration ⁷¹. Its main drawbacks are limited codec support (e.g., no MP3 encoding without a separate library) and, critically, lack of support in Safari ³².

Recommended Hybrid Architecture:

- **Recommendation: Integrate MediaBunny as the primary engine.** MediaBunny is a modern TypeScript library that intelligently combines the WebCodecs API and WASM codecs ⁷¹. When WebCodecs is available (in Chrome, Edge, Opera, Firefox), it uses it for fast, hardware-accelerated operations. When it's not available (e.g., Safari), it falls back to a WASM-based decoder. This provides the best of both worlds: maximum performance where possible and broad compatibility everywhere else. MediaBunny is small (~5KB for MP4 demuxing), has excellent TypeScript types, and is gaining traction, even being recommended by Remotion for deprecation of its own parser ⁷¹. For codecs not supported by either path (like MP3), it can seamlessly fall back to a WASM decoder, similar to how FFmpeg.wasm works ³³.

Disposition and Priorities:

- **VID-R-01: Transcoding Engine (REPLACE INTERNALLY):** The current implementation, if based on browser-native `` element manipulation, is inadequate for the required features. Replacing it with the MediaBunny-based hybrid engine is a mandatory upgrade. It passes the local-first test as it operates entirely client-side ⁷¹. **Priority: Immediate Candidate.** This is the single most important upgrade for the studio.
- **VID-U-02: Frame-Accurate Trimming (UPGRADE):** This requires seeking to an exact frame. Using `requestVideoFrameCallback` in conjunction with the WebCodecs API is the modern approach. The upgraded engine should expose this functionality. **Priority: Strong Candidate.**
- **VID-U-03: Format Conversion (UPGRADE):** As above, this is handled by the new transcoding engine. The key upgrade is supporting modern container formats and codecs efficiently. **Priority: Immediate Candidate.**
- **VID-N-04: Animated GIF, APNG, AVIF Creation (NEW CAPABILITY):** These are popular formats for web content. The upgraded engine should support creating them. Libraries like `gif-wasm` or integrations with FFmpeg.wasm's `palettegen` filter can achieve this ³³. **Priority: Strong Candidate.**

- **VID-N-05: Compress to Target File Size (NEW CAPABILITY):** This is a valuable but complex feature. It requires a loop of encoding and measuring until the target size is met. It would be a significant workload for a low-end device and must be carefully implemented with progress reporting and cancellation. **Priority: Defer.** It depends on a stable and performant core engine.
- **VID-N-06: Scene Detection (NEW CAPABILITY):** Automatically detecting scene changes can help users create clips more easily. This is a computationally expensive task involving frame differencing and could be implemented as a WASM module. **Priority: Experimental.** It depends on the maturity and stability of the underlying WASM technology for computer vision tasks.

GS-Audio (WebAudio Studio)

The WebAudio studio focuses on trimming, equalization, format conversion, and visualization. The primary opportunity for improvement lies in expanding codec support beyond the browser's bare minimum and providing more advanced audio effects.

Candidate Implementation Analysis:

- **web-audio-api-node:** This is a compelling candidate. It provides a Node.js-compatible implementation of the Web Audio API that runs in the browser via WebAssembly ³⁶. Crucially, it bundles pre-compiled decoders for MP3, WAV, FLAC, OGG, and AAC, solving the common problem of poor native support for these formats ³⁶. It uses SIMD optimizations for performance and has a permissive MIT license. This library effectively brings the full power of the Web Audio API to a wider range of audio files.
- **Tone.js / Wavesurfer.js:** These are popular JavaScript libraries built on top of the Web Audio API. Tone.js is a synthesizer and digital signal processing library, while Wavesurfer.js is primarily a waveform visualization and player component ³⁷. They are excellent for building UI and composing audio but may not solve the core codec support problem as directly as `web-audio-api-node`.

Disposition and Priorities:

- **AUD-U-01: Format Converter (UPGRADE):** The current implementation likely only supports WAV and possibly OGG/Opus. Integrating `web-audio-api-node` would add robust support for MP3, FLAC, and AAC, which are extremely common user formats. This is a high-value upgrade. **Priority: Immediate Candidate.** It directly addresses a major limitation of the Web Audio API in the browser.

- **AUD-U-02: High-Res Audio Trimmer (UPGRADE):** While the Web Audio API is good, using a dedicated library can ensure sample-accurate trimming, especially for very long files, by leveraging streaming decoding via WebCodecs. **Priority: Strong Candidate.**
- **AUD-U-03: 10-Band Equalizer (UPGRADE):** The quality of the equalizer depends on the filter type and Q factor. A dedicated DSP library can provide more control over these parameters. **Priority: Strong Candidate.**
- **AUD-N-04: Pitch-Change and Time-Stretch (NEW CAPABILITY):** The SoundStretch family of time-stretching algorithms provides high-quality pitch and tempo modification without affecting the other. Wrapping this in a WASM module would be a powerful addition for musicians and podcasters. **Priority: Strong Candidate.**
- **AUD-N-05: Noise Reduction (NEW CAPABILITY):** Real-time noise suppression models like RNNoise can be compiled to WASM and run in the browser. Integrating this would provide a valuable tool for cleaning up audio recordings. **Priority: Strong Candidate.**
- **AUD-N-06: Microphone Recording (NEW CAPABILITY):** This requires using the `MediaRecorder` API. It introduces a new permission but fits squarely within the studio's scope. The recorded audio stream would then be processed by the studio's tools. **Priority: Immediate Candidate.**

Document and Data Studio Deep Dive: PDF, EBook, Slides, Sheets, Text

This section analyzes the five studios focused on documents and data: GS-PDF, GS-EBook, GS-Slides, GS-Sheets, and GS-Text. These studios present unique challenges related to complex file formats, structured data manipulation, and rendering fidelity. The analysis focuses on upgrading existing capabilities with mature open-source libraries and identifying new features that enhance their utility as a cohesive suite for document and data processing.

GS-PDF (PDF Tools)

The PDF studio offers merging, splitting, compressing, reordering, converting, and stamping. These are complex operations on a highly structured binary format, making them ideal candidates for a dedicated, mature library rather than ad-hoc browser APIs.

Candidate Library Analysis:

- **pdf-lib:** This is a popular, pure-JavaScript library for creating and modifying PDFs in both browser and Node.js environments ⁴² ⁴³. It has a clean API for tasks like merging/splitting documents, adding pages, and placing text/images. It is actively maintained and licensed under MIT/Apache 2.0, making it very attractive. However, some reports suggest its maintenance may be 'Partially maintained' as of 2025 ⁴⁴. It excels at programmatic PDF generation and modification but may not be the best for high-fidelity rendering of arbitrary PDFs.
- **PDF.js:** Maintained by Mozilla, this is the definitive open-source PDF rendering engine for the web ⁴³ ⁴⁴. Its strength is in viewing and rendering PDFs with high fidelity, supporting text selection and searching. It cannot create or modify PDFs, so it is complementary to a library like pdf-lib rather than a replacement ⁴⁴.
- **PDFium (via WASM):** PDFium is Google's PDF engine, used by Chrome and Chromium. There are WASM builds available ⁴⁴. It is extremely powerful and supports a wide range of PDF features, including forms and annotations. However, integrating it can be complex, and its licensing is a consideration.
- **jsPDF:** This is another widely used library for generating PDFs from scratch in the browser ⁴² ⁴³. It is simpler than pdf-lib but less powerful, focusing on adding text and images with basic styling. It is good for simple reports but lacks the advanced modification capabilities of pdf-lib.

Disposition and Priorities:

- **PDF-R-01: Modification Engine (REPLACE INTERNALLY):** The current implementation is likely rudimentary. Replacing it with **pdf-lib** is the most logical step. It is a mature, well-documented library with a permissive license. It passes the local-first test. **Priority: Immediate Candidate.** It provides a solid foundation for all the listed capabilities.
- **PDF-U-02: Merge/Split/Reorder (UPGRADE):** These operations are core functionalities of pdf-lib. The upgrade would involve replacing any custom logic with calls to its ``mergeDocuments``, ``splitDocument``, and array manipulation methods. **Priority: Immediate Candidate.**
- **PDF-U-03: Compress (UPGRADE):** PDF compression typically involves downsampling images and stream filtering. pdf-lib can assist with this by allowing access to page content, but a more robust solution might require a WASM-based image processing library (from GS-Pixels) to recompress embedded images before placing them back into a new PDF. **Priority: Strong Candidate.** This is a complex feature that bridges two studios.

- **PDF-U-04: Convert (UPGRADE):** Converting PDF to image can be done by rendering the PDF page using a library like PDF.js and capturing the canvas. Converting images/text to PDF is where **pdf-lib** excels. The feasibility of Office-format conversion (DOCX, XLSX) is very low without a server; it should be classified as **DO NOT ADD**. **Priority: Strong Candidate** for PDF-to-image and text-to-PDF.
- **PDF-U-05: Stamp (UPGRADE):** Adding watermarks, page numbers, and headers/footers is a strength of pdf-lib. The upgrade would involve enhancing the UI and leveraging the library's text and image placement functions. **Priority: Immediate Candidate**.
- **PDF-N-06: True Redaction (NEW CAPABILITY):** This is a critical security feature. True redaction means permanently deleting content, not just covering it with a white rectangle. This requires parsing the PDF content streams to remove the text and image instructions. This is a highly complex feature and should be a **Strong Candidate**. It requires a deep understanding of the PDF specification.
- **PDF-N-07: Form Filling and Flattening (NEW CAPABILITY):** This is a common business need. pdf-lib has some support for form fields, but a robust solution may require a more powerful library or deeper integration. **Priority: Strong Candidate**.

GS-EBook (EPUB Studio)

The EPUB studio provides a reader and a Markdown-to-EPUB builder. EPUB is itself a ZIP archive containing HTML, CSS, and other resources, which presents both opportunities for shared engines and challenges related to security and structure.

Candidate Engine Analysis:

- **epub.js / foliate-js:** These are mature, open-source EPUB readers for the web ⁴³₆₅. They render the EPUB's HTML content and handle the spine, TOC, and pagination. They are typically licensed under permissive licenses like MIT.
- **Markdown to EPUB:** This requires a library that can take a Markdown string and generate a valid EPUB 3.0 package. The **unified/remark/rehype** ecosystem is the de facto standard for transforming Markdown. It can parse Markdown, transform the AST (Abstract Syntax Tree), and compile it to HTML. A custom transformer would be needed to bundle this HTML along with META-INF, OEBPS, mimetype, and content.opf files into a valid EPUB structure.
- **DOMPurify:** Since EPUB content can contain scripts and external references, using a sanitizer like DOMPurify is a first-class finding. It is crucial for sandboxing EPUB content and preventing XSS vulnerabilities ⁹⁴.

Disposition and Priorities:

- **EBO-U-01: Reader (UPGRADE):** The existing reader should be replaced with a mature library like **epub.js** to ensure proper rendering of EPUB 2 and 3, reflowable layouts, and table of contents navigation. **Priority: Immediate Candidate.**
- **EBO-U-02: Markdown-to-EPUB (UPGRADE):** The current implementation should be replaced with a robust pipeline using the **unified/remark/rehype** stack. This provides a flexible and powerful way to generate EPUBs from Markdown, supporting metadata, covers, and styling. **Priority: Immediate Candidate.**
- **EBO-N-03: Metadata Editing (NEW CAPABILITY):** Allowing users to edit EPUB metadata (title, author, etc.) before publication is a valuable feature. This requires parsing and rewriting the ``content.opf`` file. **Priority: Strong Candidate.**
- **EBO-N-04: Reading Other Formats (NEW CAPABILITY):** Supporting formats like FB2 or comic book archives (CBZ/CBR) would expand the studio's utility. CBZ is just a ZIP of images, so it could be handled by **GS-Archive**. FB2 would require a dedicated parser. **Priority: Investigate Further.** Licensing of parsers for these formats must be checked.
- **EBO-D-05: DRM Circumvention (DO NOT ADD):** Any capability related to circumventing Digital Rights Management is unethical and illegal. It must be explicitly rejected. **Reason:** Violates copyright law and ethical guidelines.
- **EBO-B-06: Boundary with GS-Archive:** Since EPUB, CBZ, and CBR are all ZIP-based formats, the engine from **GS-Archive** should be shared. The EBook studio would contain the logic for interpreting the specific structure of these formats (e.g., ``toc.ncx`` for EPUB, ``mimetype`` file in PKZIP archives).

GS-Slides (Deck Studio)

The Slides studio converts Markdown to a presenter view and generates PDF handouts. Its primary challenge is rendering Markdown with layout and typography appropriate for slides.

Candidate Engine Analysis:

- **Marp/Marpit:** Marp is a popular tool that renders Markdown with special syntax into beautiful presentations and PDFs. It has a CLI, VS Code plugin, and a browser-based version. It is known for its elegant output and tight integration between slide deck and PDF export. It is a strong candidate for the rendering engine.
- **reveal.js:** This is a mature and highly extensible framework for creating HTML-based presentations ⁷⁹. It is the backbone of many presentation tools and has excellent support for speaker notes, themes, and transitions. It could serve as the rendering engine.

- **Typst WASM:** Typst is a modern, programmable document compiler that can generate PDFs. A WASM build would allow it to run in the browser, potentially offering a very high-quality, typesetting-focused alternative for generating the PDF handouts [42](#) [44](#).

Disposition and Priorities:

- **SLI-U-01: Markdown Rendering (UPGRADE):** The current renderer should be replaced with a more robust and feature-rich engine like **Marp** or **reveal.js**. Marp is particularly well-suited as it is designed for this exact workflow. **Priority: Immediate Candidate.**
- **SLI-U-02: Presenter View (UPGRADE):** Both Marp and reveal.js have built-in presenter views. The upgrade involves adopting one of these engines. **Priority: Immediate Candidate.**
- **SLI-N-03: Themes and Templates (NEW CAPABILITY):** Allowing users to create and share themes would be a powerful feature. This would require a templating engine and a mechanism for storing/loading theme definitions. **Priority: Strong Candidate.**
- **SLI-N-04: PPTX Export (NEW CAPABILITY):** Exporting to PowerPoint is a common request. This is technically challenging as it requires generating a complex XML-based ZIP archive. A library like **pptx-viewer-core** [78](#) or **PptxGenJS** [80](#) could be used. **Priority: Defer.** This is a high-complexity feature that may be out of scope initially.
- **SLI-B-05: Boundary with GS-EBook and GS-Text:** The Markdown parsing stage should be shared between **GS-Slides**, **GS-EBook**, and **GS-Text**. Using the **unified/remark/rehype** stack across all three studios would ensure consistency and prevent code duplication. This is a key platform-level recommendation.

GS-Sheets (CSV & Data Grid)

The Sheets studio is the most complex, dealing with parsing, rendering, and manipulating tabular data. The choice of library is critical for performance, especially with large datasets.

Candidate Library Analysis:

- **SheetJS (xlsx):** With millions of weekly downloads, SheetJS is the dominant library for reading/writing spreadsheet files in JavaScript [68](#). Its Community Edition (Apache 2.0) is excellent for parsing CSV, XLSX, and other formats and

converting them to JSON. Its main weakness is that its default read operation loads the entire workbook into memory, making it unsuitable for very large files ⁶⁸.

- **ExcelJS**: This library is known for its superior streaming capabilities for both reading and writing, maintaining constant memory usage regardless of file size ⁶⁸. It has a cleaner object-oriented API and is better suited for serverless environments. Its main drawback is a larger bundle size and stricter error handling ⁶⁸.
- **hucre**: This is a newer, zero-dependency TypeScript library that supports streaming and large files (>4GB with ZIP64) ⁶⁹. It is MIT licensed and very lightweight, making it a compelling modern alternative.
- **HyperFormula**: This is a headless spreadsheet calculation engine compatible with Excel formulas ⁵⁹. It is dual-licensed (GPLv3 and commercial), which poses a significant licensing hurdle for a permissively licensed project like GS Softwares ⁵⁹.

Disposition and Priorities:

- **SHT-R-01: Data Parsing Engine (REPLACE INTERNALLY)**: Given the issues with SheetJS's memory usage, a switch to **ExcelJS** or **hucre** is warranted. **ExcelJS** is the stronger choice due to its mature streaming API and focus on styled workbooks ⁶⁸. **hucre** is a very promising lightweight alternative. **Priority: Immediate Candidate.**
- **SHT-U-02: Grid Performance (UPGRADE)**: For displaying hundreds of thousands of rows, a virtualized grid is essential. Libraries like **TanStack Table/Virtual** or **AG Grid** (Community Edition) should be used. AG Grid's enterprise features like formula support come at a cost ⁶⁰. **Priority: Immediate Candidate.**
- **SHT-N-03: Formula Evaluation (NEW CAPABILITY)**: This is a massive undertaking. Instead of building a formula engine from scratch, the platform should consider integrating a headless engine like **HyperFormula**. However, its GPL license is a major blocker. A more feasible approach would be to start with a smaller set of common functions (SUM, AVERAGE, IF) and build up. **Priority: Defer.** Requires careful licensing consideration.
- **SHT-N-04: SQL Over Local Files (NEW CAPABILITY)**: Running SQL queries on local CSV or Excel files is a powerful feature. A WASM port of SQLite, like **AbsurdSQL** or the official **sqlite-wasm** package, would be perfect for this. It provides a full SQL engine in the browser ⁶ ⁸⁴. **Priority: Strong Candidate.**
- **SHT-N-05: Join/Pivot Tables (NEW CAPABILITY)**: These analytical features go beyond a simple grid. They could be implemented as higher-level components that operate on the data provided by the parsing engine. **Priority: Defer.** This adds significant complexity.

GS-Text (Code & Diff)

The Text studio contains a diff comparator, formatter, regex tester, and Base64 encoder. These are text-centric utilities that can be significantly enhanced with mature open-source tools.

Candidate Tool Analysis:

- **Diff Comparison:** `jsdiff` or `diff-match-patch` are robust libraries for computing differences between texts. **Monaco Diff Editor** (from VS Code) provides a high-quality UI for this.
- **JSON/YAML Formatting:** `jsonc-parser` is a fast, native-engine-powered parser for JSON and JSONC. `js-yaml` is a standard library for YAML. **Prettier** is the gold standard for code formatting and could be compiled to WASM.
- **Regex Testing:** The browser's native `RegExp` engine is sufficient, but for more advanced engines like RE2 (which is resistant to catastrophic backtracking), a WASM build would be beneficial. **RE2 WASM builds** exist.
- **Base64 Encoding:** This is a standard library function and does not need a third-party library.

Disposition and Priorities:

- **TXT-U-01: Diff Comparator (UPGRADE):** Replace any custom diffing logic with a mature library like `jsdiff`. Add features like word-level granularity and moved-block detection. **Priority: Immediate Candidate.**
- **TXT-U-02: JSON/YAML Formatter (UPGRADE):** Use `jsonc-parser` and `js-yaml` for accurate parsing and formatting. Add support for comments (JSONC) and anchors (YAML). **Priority: Immediate Candidate.**
- **TXT-N-03: Rich Text Editor (NEW CAPABILITY):** Adding a code editor component like **CodeMirror 6** or **Monaco Editor** would be a significant expansion. It must be lightweight, accessible, and have a permissive license. **Priority: Strong Candidate.**
- **TXT-N-04: jq-style Querying (NEW CAPABILITY):** Compiling `jq` to WASM would allow users to query and transform JSON data directly in the browser. **Priority: Strong Candidate.**
- **TXT-B-05: Boundary with GS-Hash and GS-Security:** Hashing (SHA-256, MD5) belongs in **GS-Hash**. Encryption should be handled by **GS-Security**. The text studio should not become a grab bag of unrelated utilities. The domain-fit test must be applied strictly.

Utility and Integrity Studio Deep Dive: Archive, QR & Barcode, Security, Hash

This section examines the four utility and integrity-focused studios: GS-Archive, GS-QR & Barcode, GS-Security, and GS-Hash. These tools form the bedrock of file manipulation, secure data handling, and data verification. The analysis emphasizes replacing weak or incomplete implementations with robust, mature open-source libraries, ensuring safety and reliability in their operations.

GS-Archive (ZIP & TAR)

The Archive studio is tasked with inspecting, compressing, extracting, and converting multi-file archives. Handling archives safely is paramount, as they can contain malicious payloads or cause decompression bombs.

Candidate Library Analysis:

- **fflate**: This is a pure-JavaScript library noted for its speed and small bundle size (~8kB base package) ⁹⁷. It provides fast DEFLATE, GZIP, Zlib, and ZIP implementations. Its asynchronous API uses Web Workers to avoid blocking the main thread, and it supports streaming for both creation and extraction, making it suitable for large files ^{97 99}. It is MIT licensed.
- **zip.js**: This is a well-established library for compressing and decompressing zip files, licensed under BSD-3-Clause ⁴⁶. It supports large archives (ZIP64) and progressive creation, making it a solid choice ^{45 46}.
- **Libarchive via WASM**: Libarchive is a command-line utility for handling various archive formats. WASM builds exist that wrap it, potentially offering support for 7z, gzip, bzip2, and others ⁴⁵. However, its licensing and size are concerns.
- **pako**: A classic, stable port of zlib, it is good for GZIP/DEFLATE but does not support ZIP file creation/extraction itself ⁹⁸.

Disposition and Priorities:

- **ARC-R-01: Compression/Extraction Engine (REPLACE INTERNALLY)**: The current implementation is likely inadequate. **fflate** is the strongest candidate due to its performance, streaming support, worker-based async model, and small size ⁹⁷. **zip.js** is a viable alternative. **Priority: Immediate Candidate.**

- **ARC-U-02: Inspection (UPGRADE):** Reading the central directory without extracting is a key safety feature. Both **fflate** and **zip.js** support this. The upgrade involves implementing this feature. **Priority: Strong Candidate.**
- **ARC-U-03: Safety Features (UPGRADE):** The implementation must include defenses against "zip slip" (path traversal) and decompression bombs (resource exhaustion). These are first-class findings and must be implemented using the chosen library's safe APIs. **Priority: Immediate Candidate.**
- **ARC-N-04: New Archive Formats (NEW CAPABILITY):** Expanding to support 7z, gzip, bzip2, xz, and zstd would be valuable. This is a significant undertaking. A modular approach where new format support is added as separate WASM modules would be best. **Priority: Defer.** Depends on the modularity of the chosen engine.
- **ARC-N-05: Per-Entry Checksums (NEW CAPABILITY):** Computing checksums for individual entries inside an archive is a useful forensic and integrity-checking feature. This would be a natural extension of the inspection capabilities. **Priority: Strong Candidate.** Should reuse the hashing engine from **GS-Hash**.

GS-QR & Barcode (QR & Barcode Studio)

The QR & Barcode studio generates QR codes and other symbologies. The key challenges are ensuring scannability of branded codes, supporting a wide range of formats, and correctly handling payload syntax.

Candidate Library Analysis:

- **qrcode-generator:** A compact, popular library for generating QR codes, licensed under MIT ⁵⁷. It gives low-level control over the raw modules, which is useful for creating custom-styled codes. It supports all versions and error correction levels.
- **html5-qrcode:** This is a mature library for client-side QR and barcode SCANNING, running entirely in the browser using the camera ⁵⁶. It supports a wide range of symbologies and handles browser quirks, especially on iOS Safari.
- **ZXing-based WASM Decoders:** The ZXing project has Java and C++ implementations that can be compiled to WASM. These offer very broad symbology support but may be larger in size.
- **bwip-js / JsBarcode:** These are versatile libraries for generating a wide variety of barcodes (Code 128, ITF, etc.) ⁵⁸.

Disposition and Priorities:

- **QRB-U-01: QR Generator (UPGRADE):** The current generator should be replaced with **qrcode-generator** for its permissive license and low-level control over the

output ⁵⁷. This allows for better implementation of branded QR codes. **Priority: Immediate Candidate.**

- **QRB-U-02: Branded QR Codes (UPGRADE):** The upgrade must focus on the trade-offs: logos can interfere with error correction. The implementation should provide guidance on contrast, quiet zones, and logo size. It must include a round-trip scannability test. **Priority: Strong Candidate.**
- **QRB-N-03: QR/Barcode Scanning (NEW CAPABILITY):** Adding a scanner would be a major new feature. `html5-qrcode` is the clear choice due to its active maintenance and broad format support ⁵⁶. **Priority: Strong Candidate.** It requires camera permissions and runs in a secure context (HTTPS).
- **QRB-N-04: Additional Symbologies (NEW CAPABILITY):** The studio should be expanded to support more symbologies like Aztec, PDF417, Data Matrix, and GS1 variants. This can be achieved by integrating **ZXing-based WASM decoders** or other specialized libraries. **Priority: Strong Candidate.**
- **QRB-D-05: Dynamic QR Codes (DO NOT ADD):** Dynamic QR codes that redirect to a server or track scans violate the local-first model. They require a backend. **Reject** because they introduce server dependency and telemetry.
- **QRB-B-06: Boundary with GS-Bridge:** A scanning result (a decoded string) should be treated as a piece of text data and passed to other studios via GS-Bridge. The QR/Barcode studio's role is to perform the scan, not to interpret the payload.

GS-Security (AES-256)

This studio is the most sensitive, requiring conservative treatment and adherence to cryptographic best practices. It implements PBKDF2 for password-based key derivation and AES-256-GCM for file encryption.

Analysis and Recommendations:

- **PBKDF2 and Key Derivation:** The implementation should use the browser's native **SubtleCrypto** API for PBKDF2. The number of iterations must follow current NIST/OWASP guidelines (e.g., thousands of iterations). The library `@noble/hashes` provides an audited, high-security implementation of PBKDF2 that can be used to verify the correctness of the native implementation ⁹³.
- **AES-256-GCM:** This should be implemented using **SubtleCrypto.encrypt()** and **SubtleCrypto.decrypt()**. The main challenge is handling large files, as Web Crypto's digest operations are one-shot. The solution is to use chunked authenticated encryption, processing the file in blocks.
- **Encrypted File Format:** A custom, documented file format should be created. It should include magic bytes, a version field, an authenticated header containing the

salt and iteration count for PBKDF2, and then the encrypted data in chunks. This ensures interoperability.

- **Alternatives to PBKDF2:** Argon2id is a modern, more memory-hard KDF. Audited WASM implementations exist (e.g., `hash-wasm`). The benefit in terms of security against GPU cracking is significant, but it adds a dependency. **libsodium.js** is another excellent option, providing a high-level, easy-to-use cryptography wrapper. **Priority: Investigate Further.** The trade-off between simplicity (native PBKDF2) and security (Argon2) needs evaluation.
- **Threat Model:** The interface must clearly warn the user that JavaScript cannot reliably wipe memory. Passwords should never be echoed to the console. The threat model is against passive eavesdropping on the file, not against an attacker with access to the user's machine while the app is running.

Disposition and Priorities:

- **SEC-U-01: PBKDF2 Implementation (UPGRADE):** Verify that the current implementation uses the native `SubtleCrypto` API with a sufficiently high iteration count. Reference current NIST/OWASP guidelines. **Priority: Immediate Candidate.**
- **SEC-U-02: Large File Encryption (UPGRADE):** The current implementation likely fails for large files. It must be upgraded to use a chunked encryption scheme. **Priority: Immediate Candidate.**
- **SEC-N-03: Public-Key Cryptography (NEW CAPABILITY):** The Web Crypto API now supports X25519 (for key exchange) and Ed25519 (for signatures). Adding support for these would be a valuable upgrade. **Priority: Strong Candidate.**
- **SEC-N-04: Self-contained Decrypt Page (NEW CAPABILITY):** Creating a simple, self-contained HTML page that allows anyone with the password to decrypt a file is a great usability feature. **Priority: Strong Candidate.**
- **SEC-B-05: Boundary with GS-Hash:** HMAC is a cryptographic hash function. It belongs in **GS-Security**, not **GS-Hash**, as its primary use case is authentication, not data integrity verification. The domain-fit test favors **GS-Security**.

GS-Hash (Checksum & Verify)

The Hash studio computes checksums for integrity verification. The main upgrade opportunity is to support hashing of large files efficiently and to add more modern and non-cryptographic algorithms.

Candidate Library Analysis:

- **@noble/hashes:** This is an audited, high-security, tree-shakeable library for a wide range of hash functions (SHA-2, SHA-3, Blake3, MD5, SHA-1, etc.) ⁹³. It provides a streaming interface, which is essential for hashing large files without loading them entirely into memory. It is written in TypeScript and has excellent documentation.
- **Web Crypto API:** The native `crypto.subtle.digest()` method supports SHA-1 and the SHA-2 family. It is a good option for these algorithms but does not support MD5 and lacks a streaming API, meaning it can only hash data that fits in memory ⁹⁶.
- **spark-md5:** A pure-JavaScript library for MD5. It is simple but likely slower than a WASM-based implementation.

Disposition and Priorities:

- **HAS-U-01: Large File Hashing (UPGRADE):** The current implementation likely fails for large files. The upgrade is to implement a streaming hasher. **@noble/hashes** is the perfect fit. **Priority: Immediate Candidate.**
- **HAS-U-02: Algorithm Coverage (UPGRADE):** The implementation should unify behind **@noble/hashes** to support all relevant algorithms (SHA-256, SHA-512, SHA-1, MD5, SHA-384, SHA-3, BLAKE2, BLAKE3) in a single, consistent engine. **Priority: Immediate Candidate.**
- **HAS-N-03: Non-Cryptographic Checksums (NEW CAPABILITY):** Adding CRC32, Adler-32, and xxHash would be useful for developers and publishers who use these for performance-sensitive tasks. **xxHash** has excellent WASM implementations. **Priority: Strong Candidate.**
- **HAS-N-04: Manifest Verification (NEW CAPABILITY):** A feature to compare a computed hash list against a published manifest file (e.g., `sha256sum.txt`) would be a valuable addition. **Priority: Strong Candidate.**
- **HAS-N-05: Credential Hashing (NEW CAPABILITY):** Tools for generating bcrypt or Argon2 hashes could be useful for developers. This blurs the line with **GS-Security**. Given its developer focus, it could reside here, but a shared engine for these algorithms would be better. **Priority: Strong Candidate.**

Orchestration and Platform-Wide Infrastructure Architecture

To elevate GS Softwares from a collection of tools to a cohesive platform, a robust orchestration layer and shared infrastructure are essential. This section details the architecture for **GS-Bridge**, the central orchestrator, and outlines the critical shared platform components required to enable seamless, multi-studio workflows while adhering to the local-first principles.

GS-Bridge: The Platform-Level Workflow Engine

After analyzing the options, the conclusion is that **GS-Bridge should become a platform-level workflow engine**. Treating it as merely another utility suite would underutilize its potential. A platform-level engine is necessary to manage complex, multi-step pipelines that span multiple studios, abstracting away the complexity of data transfer, resource management, and execution flow.

Proposed Architecture:

1. **Typed Asset Taxonomy:** The foundation of the engine is a system for representing assets. An asset should be more than a raw `File` object. It should be a handle to a piece of data stored in the shared workspace, enriched with metadata:
 - ``WorkspaceAsset { id: string, handle: FileSystemFileHandle, mimeType: string, size: number, hash: string, lineage: [string], createdAt: Date }``
 - Specialized types like ``WorkspaceImage``, ``WorkspaceVideo``, ``WorkspaceAudio``, and ``WorkspaceText`` would inherit from this base type, providing studio-specific properties and methods. This typing is crucial for validating connections within a workflow.
1. **Workflow Graph Representation:** A workflow is a directed acyclic graph (DAG) of steps. Each step represents a single capability from a specific studio. The graph would be serialized into a JSON format, making it portable and versionable. A step definition might look like:

```
{
  "id": "step-1",
  "toolId": "gs-pixels/resize",
  "inputs": {
    "image": "asset-uuid-from-previous-step",
```

```

    "width": 800,
    "height": 600
  }
}

```

This declarative format allows for template creation and parameterization.

2. **Intermediate Representations and Zero-Copy Transfer:** To maximize performance and minimize memory usage, the engine must facilitate zero-copy data transfer between steps. For image/video/audio frames, this means passing `ImageBitmap`, `VideoFrame`, or `AudioData` objects directly between Web Workers. For larger files, the engine should spill intermediate data to OPFS and pass around file handles. `Transferable Objects` are the key mechanism for moving large arrays between threads efficiently ¹⁹.
3. **Pipeline Execution Manager:** This is the core of GS-Bridge, running in a dedicated Web Worker. It is responsible for:
 - **Scheduling:** Managing a queue of jobs and assigning them to a pool of worker threads.
 - **Dependency Resolution:** Loading required WASM engines and dependencies for each step on-demand (lazy loading).
 - **Resource Budgeting:** Enforcing memory and CPU limits per job to prevent the browser tab from crashing. It should monitor peak memory usage and gracefully fail or pause jobs that exceed their budget ³³.
 - **Progress and Cancellation:** Propagating ``AbortSignal`` and progress events through the entire pipeline. Aggregating progress from parallel steps.
 - **Failure and Retry:** Implementing idempotent steps where possible. Providing mechanisms for retrying failed steps or resuming from checkpoints.
1. **The OCR and Text Axis:** Recognizing that OCR and speech-to-text are content-understanding capabilities, they should be implemented as a shared engine. A WASM port of a model like `Whisper.cpp` ³¹ for speech-to-text and a library like `tesseract.js` ⁴⁴ for OCR would be hosted by the execution manager. A new node type, `RecognizeTextStep`, would consume an image or video and produce text, which could then be passed to other studios.

High-Value Workflow Catalog (Prioritized):

Workflow Name	Description	Steps	Resource Profile	Local-First Status	Disposition & Priority
Create Social Media Post	Take a photo, apply a filter, add a caption, and export as a square image and a vertical video clip.	1. GS-Pixels (Crop to Square) 2. GS-Pixels (Apply Filter) 3. GS-Slides (Markdown to Image)	Low-Medium RAM/CPU	Yes	Immediate Candidate
Generate Presentation Assets	Take a long video, extract key frames as thumbnails, and create a slide deck with the frames and a transcript.	1. GS-Video (Extract Frames) 2. GS-Audio (Extract Audio) 3. Whisper.cpp (Speech-to-Text) 4. GS-Slides (Markdown to Deck)	Medium-High RAM/CPU	Yes	Strong Candidate
Prepare Document for Sharing	Take a multi-page scanned PDF, run OCR to make it searchable, and save as a new PDF.	1. GS-PDF (Split Pages) 2. Tesseract.js (OCR each page) 3. GS-PDF (Merge with text layer)	High RAM/CPU	Yes	Strong Candidate
Create Animated GIF from Video	Trim a video, convert to an optimized GIF with a reduced palette.	1. GS-Video (Trim) 2. MediaBunny (Convert to GIF)	Medium RAM/CPU	Yes	Immediate Candidate
Verify Download Integrity	Hash a downloaded file and compare it against a provided checksum manifest.	1. GS-Hash (Compute Hashes) 2. GS-Hash (Compare against Manifest)	Low RAM/CPU	Yes	Immediate Candidate

Shared Platform Infrastructure

The following platform-wide components are necessary to support the evolution of the 14 studios and the GS-Bridge engine.

Shared Workspace and Global Asset Store:

- **Technology:** The recommended architecture is **OPFS-backed SQLite WASM** ⁸ ⁸⁴. OPFS provides the high-performance, durable file I/O needed for storing large media files and temporary assets ⁸⁶. SQLite provides a structured, relational database to store metadata about these assets (type, size, hash, creation date, lineage), user preferences, and workflow definitions.
- **Functionality:** The store should allow importing a file once and then referencing it from any studio. It must handle storage quotas, persistence requests, and eviction policies, noting that Safari is known to be aggressive in clearing storage ³. Cross-tab consistency can be managed using the **BroadcastChannel API** ¹⁸.

Tool Registry and SDK:

- **Tool Registry:** A central metadata schema is needed to describe every capability. This registry drives discovery, navigation, and workflow creation. A capability registration might look like:


```

```typescript
interface ToolDefinition {
 id: 'gs-pixels/resize'; // Unique ID
 studio: 'GS-Pixels';
 name: 'Resize';
 description: 'Resize an image to the specified dimensions.';
 inputTypes: ['WorkspaceImage'];
 outputType: 'WorkspaceImage';
 parameters: { width: { type: 'number', default: 100 }, height: { type: 'number', default: 100 } };
 version: '1.0.0';
 // ... provenance info
}
```

```

- **Tool SDK:** Every studio must implement a common contract. This SDK defines the lifecycle (`initialize`, `validate`, `run`, `cancel`, `dispose`), typed inputs/outputs, progress/event emitters, and error model. This contract is what allows the execution manager to run any tool in a predictable way ⁸.

Platform Shell and UI/UX Design System:

- **Shell:** A single-page application shell is needed to manage routing and application state. A lightweight framework or a framework-free approach using web components could be used. The shell would be responsible for lazy-loading studios and capabilities as needed to optimize initial load time.
- **Design System:** A shared design system with design tokens, accessible headless UI primitives, and consistent patterns for file drop, progress bars, and errors will ensure a cohesive user experience across all 14 studios.

Permissions and Security Model:

- **Permission Declaration:** Each tool must declare its permissions needs (e.g., `['files', 'workspace']`). The platform should enforce these in a browser-safe manner.
- **Isolation:** Third-party code should be isolated in sandboxed frames or workers. A strict Content Security Policy (CSP) is mandatory. Subresource Integrity (SRI) must be used for all externally loaded scripts ³⁸.

Versioning and Provenance:

- **Versioning:** A comprehensive versioning strategy is needed for tool schemas, dependency versions, and asset/workflow schemas to manage migrations and ensure backward compatibility ⁸ .
- **Provenance and SBOM:** The platform must maintain a Software Bill of Materials (SBOM). An automated process should be in place to generate this SBOM and display it to the user. This can be achieved using tools like Syft and Gype in a CI/CD pipeline ^{38 54} . License compatibility must be checked for all dependencies, especially when combining libraries with different licenses ⁴⁸ .

By implementing this robust orchestration layer and shared infrastructure, GS Softwares can achieve its goal of becoming a truly integrated and powerful local-first platform, capable of handling complex, multi-step creative and data-processing tasks entirely within the user's browser.

Implementation Roadmap, Open-Source Matrix, and Final Risks

This final section synthesizes the research findings into a practical implementation roadmap, presents a consolidated matrix of recommended open-source technologies, and details the remaining risks and unknowns that require attention before proceeding with development. The roadmap is designed to be phased, starting with foundational upgrades and progressing toward the more complex orchestration features, thereby managing risk and demonstrating value incrementally.

Phased Implementation Roadmap

The evolution of GS Softwares should follow a deliberate, multi-phase plan. This approach allows the team to build a solid foundation before tackling more complex, interconnected features.

- **Phase 1: Foundation and Core Studio Upgrades (Months 1-6)**
- **Objective:** Modernize the core technical foundation and upgrade the most frequently used studios.
- **Key Actions:**

1. Establish the **Shared Workspace** using the OPFS + S
 2. Build the **Tool SDK and Common Contract**. Define th
 3. Upgrade **GS-Pixels** using a WASM engine like **stron**
 4. Upgrade **GS-Audio** using a library like **web**
 5. Upgrade **GS-Text**'s diff and formatter capabilities
 6. Upgrade **GS-Hash**'s large-file hashing with **strong**
- **Outcome:** A faster, more capable version of the platform with a modernized tech stack. Users immediately benefit from improved performance and new features in the core media and utility tools.
- **Phase 2: Advanced Studio Features and Interoperability (Months 7-12)**
 - **Objective:** Expand the feature set of key studios and begin establishing the boundaries for cross-studio interaction.
 - **Key Actions:**
 1. Upgrade **GS-Video** with the **MediaBunny**/**str**
 2. Upgrade **GS-PDF** with the **pdf-lib**/**strong**
 3. Implement **GS-Archive** with **fflate**/**strong**
 4. Begin developing the **GS-Bridge** concept, starting
 5. Define the shared **Markdown parsing/rendering engine**/**stron**
 6. Implement the first **cross-studio boundary rules**/**strong**,
 - **Outcome:** Studios become significantly more powerful. The groundwork for true interoperability is laid, and the platform begins to feel more like an integrated suite.
- **Phase 3: Full Orchestration and Complex Workflows (Months 13-18)**
 - **Objective:** Launch the full GS-Bridge orchestration engine and enable complex, multi-step workflows.
 - **Key Actions:**
 1. Fully develop the **GS-Bridge** platform-level engine
 2. Implement the **typed asset taxonomy** to ensure type
 3. Develop high-value, end-to-end **workflow examples**/**strong**
 4. Integrate the shared **OCR engine** (e.g., **te**/**strong**)
 5. Build a user-friendly **workflow editor** and gallery
 - **Outcome:** The platform transforms into a powerful, interconnected production environment. Users can automate complex, multi-stage processes, unlocking the full potential of the local-first model.

Open-Source Implementation Matrix

The following table summarizes the recommended open-source technologies for the highest-priority upgrades and new capabilities identified in this report.

| Capability | Recommended Project | Repository/ Package | Exact License(s) | Maturity & Notes | Browser Support | WASM Status & Features | Dependencies | Integration Complexity |
|-----------------------------------|---------------------|---|------------------|--|------------------------------|------------------------|---------------|------------------------|
| Image Resizing/ Processing | Photon | @silvia-odwyer/ photon | Apache-2.0 | Mature, high performance, active 112 | All modern browsers | Yes, Rust -> WASM | None | Low |
| Video Transcoding | MediaBunny | media-bunny | MPL-2.0 | Modern, high performance 71 | Chrome, Edge, Firefox, Opera | No, uses WebCodecs | None | Low |
| Audio Codec Support | web-audio-api-node | webaudio-node | MIT | High performance, WASM-based decoders 36 | All modern browsers | Yes, C++ -> WASM | SDL2, dr_libs | Medium |
| PDF Modification | pdf-lib | pdf-lib | MIT / Apache-2.0 | Mature, actively maintained 42 | All modern browsers | No | None | Low |
| Archive Handling | fflate | fflate | MIT | Very fast, small, streaming 97 | ES3+ | No | None | Low |
| Large File Hashing | @noble/hashes | @noble/hashes | MIT (audited) | Highly secure, streaming, tree-shakeable 93 | All modern browsers | No | None | Low |
| Vector Canvas Rendering | Konva | konva | MIT | Most popular, performant scene graph 106 | All modern browsers | No | None | Low |
| Boolean Path Operations | Clipper2 | @asymmetrik/ clipper-js | MPL-2.0 | High-performance, WASM-ready 111 | All modern browsers | Yes | None | Medium |
| Markdown Transformation | unified.js | unified, remark-parse, rehype-stringify | MIT | De facto standard for Markdown processing 62 | All modern browsers | No | Various | Low |
| SQLite Database | sqlite-wasm | @sql.js/ sql.js | Unlicense | Mature, embeddable SQL engine 84 | All modern browsers | Yes, C -> WASM | None | Medium |

Risks and Unknowns

The successful implementation of this research depends on addressing several key risks and unknowns.

1. **Unverified Existing Implementation:** The biggest uncertainty is the actual technology stack used in the existing 14 studios. Without observing the live site, upgrade recommendations are based on assumptions. **Action Required:** Send the information request to the owner for a list of libraries, supported formats, and known file-size limits. This will validate or invalidate all the "UPGRADE" and "REPLACE" findings.
2. **Dependency Licensing and Copyleft Obligations:** Several powerful libraries, particularly those wrapping C/C++ code like FFmpeg (`ffmpeg.wasm`), Ghostscript, and parts of libarchive, carry strong copyleft licenses (GPL/LGPL) ³³ ⁴⁴. While distributing a WASM binary is often considered a "transmission" exempt from source-code disclosure, this is a legal gray area. **Action Required:** Engage legal counsel to conduct a thorough review of all recommended dependencies with non-permissive licenses. This is a critical risk that must be formally assessed.
3. **Browser Compatibility and Graceful Degradation:** The proposed architecture relies heavily on modern APIs like WebCodecs (not on Safari ³²) and Shared Workers (not on Android Chrome ⁹²). Relying on these features creates a fragile dependency. **Action Required:** Design a robust fallback strategy. For WebCodecs, this means falling back to slower software codecs via a WASM library. For Shared Workers, this means using BroadcastChannel for messaging, accepting that it has no message history and no guaranteed delivery.
4. **Performance on Low-End Devices:** The platform's reliance on multiple WASM engines and a database will increase memory and CPU usage. The defined low-end mobile baseline (3-4 GB RAM) is a significant constraint ¹⁷. Video transcoding and large spreadsheet manipulation could easily exhaust this memory. **Action Required:** Implement aggressive memory monitoring and throttling in the execution manager. For large files, the UI must clearly communicate resource constraints and potentially block certain operations.
5. **Increased Maintenance Burden:** Adopting a complex stack of multiple WASM libraries, a database engine, and a sophisticated orchestration layer will significantly increase the long-term maintenance overhead. The team will be

responsible for keeping all these dependencies updated, patched, and compatible.

Action Required: Invest in a robust CI/CD pipeline with automated dependency updates and security scanning (SCA tools like Snyk or FOSSA) ⁴⁰. Plan for dedicated engineering time to manage this growing complexity.

By proactively addressing these risks and following the phased roadmap, the GS Softwares team can successfully navigate the technical challenges and realize the vision of a powerful, privacy-preserving, and interconnected local-first platform.

Reference

1. Local-first software: You own your data, in spite of the cloud <https://www.inkandswitch.com/essay/local-first/>
2. What is Local-first Software? <https://mikezornek.com/posts/2025/2/what-is-local-first-software/>
3. Why Local-First Software Is the Future and its Limitations <https://rxdb.info/articles/local-first-future.html>
4. Local-first Software <https://www.inkandswitch.com/local-first-software/>
5. Local-first software https://en.wikipedia.org/wiki/Local-first_software
6. Local-first Software <https://joodalooop.com/local-first/>
7. Local-first software: you own your data, in spite of the cloud <https://blog.acolyer.org/2019/11/20/local-first-software/>
8. The Architecture Of Local-First Web Development <https://www.smashingmagazine.com/2026/05/architecture-local-first-web-development/>
9. alexanderop/awesome-local-first <https://github.com/alexanderop/awesome-local-first>
10. Entry-Level Smartphones Market Size, Share, and Trends ... <https://www.databridgemarketresearch.com/reports/global-entry-level-smartphones-market>
11. The State of WebAssembly – 2025 and 2026 <https://platform.uno/blog/the-state-of-webassembly-2025-2026/>
12. Best Performance Browser Android: A Practical ... <https://webeyez.com/insights/guides/best-performance-browser-android>
13. Web Games Tech Stack in 2026 <https://app.cinevva.com/guides/web-games-stack-2026>

14. 21 Best Android Browsers for 2026, Beyond Chrome <https://www.testmuai.com/blog/best-android-browsers/>
15. Asm.js faster than WebAssembly on Android Chrome? <https://discussions.unity.com/t/asm-js-faster-than-webassembly-on-android-chrome/750368>
16. The 4 Best Android Phones of 2026 | Reviews by Wirecutter <https://www.nytimes.com/wirecutter/reviews/best-android-phone/>
17. Justifying why your budget phone is now entry-level <https://inquisitiveuniverse.com/2025/07/19/justifying-why-your-budget-phone-is-now-entry-level/>
18. Broadcast Channel API - MDN Web Docs https://developer.mozilla.org/en-US/docs/Web/API/Broadcast_Channel_API
19. Web Workers: Parallel Processing in the Browser <https://medium.com/@artemkhrenov/web-workers-parallel-processing-in-the-browser-e4c89e6cad77>
20. The BroadcastChannel API: Real-Time Tab ... <https://dev.to/itxshakil/broadcastchannel-api-a-hidden-gem-for-web-developers-33c4>
21. BroadcastChannel for Cross-Boundary Communication <https://stevekinney.com/courses/enterprise-ui/broadcast-channel>
22. Top CRDT Libraries for Real-Time Data Sync Updated July ... <https://velt.dev/blog/top-crdt-libraries-for-real-time-data-sync>
23. yjs/yjs: Shared data types for building collaborative software <https://github.com/yjs/yjs>
24. CRDTs for mobile sync: Automerge vs Yjs vs crsqlite <https://mvpfactory.io/blog/crdts-for-offline-first-mobile-sync-automerge-vs-yjs-merge-semantics-and-the/>
25. How we used module federation to implement micro ... <https://dev.to/kerryconvery/module-federation-learnings-37oi>
26. Photon: A WebAssembly Image Processing Library <https://silvia-odwyer.github.io/photon/>
27. StanleyMasinde/wasm-image-processor <https://github.com/St StanleyMasinde/wasm-image-processor>
28. Client-Side Media Processing in WordPress 7.1 <https://make.wordpress.org/core/2026/07/22/client-side-media-processing-in-wordpress-7-1/>
29. WebAssembly in 2026: Why Every Web Developer Sh... <https://coderfile.io/blog/webassembly-web-developers-2026>
30. Full FFmpeg in Browser, Offline PWA, No Uploads(WASM) <https://news.ycombinator.com/item?id=48404224>
31. tejaswigowda/ffmpeg-webCLI: A browser-based video ... <https://github.com/tejaswigowda/ffmpeg-webCLI>

32. Experimental webcam filters with FFmpeg.wasm and ... <https://transloadit.com/devtips/real-time-video-filters-in-browsers-with-ffmpeg-and-webcodecs/>
33. The complete guide to ffmpeg.wasm in 2026 - ClearRec <https://www.clearrec.app/blog/ffmpeg-wasm-complete-guide-2026>
34. Building Offline-First Web Audio Applications — Music Deeper <https://musicdeeper.com/offline-web-audio.html>
35. Browser-native Web Audio API for WASM applications. - GitHub <https://github.com/neodsp/web-audio>
36. webaudio-node - npm <https://www.npmjs.com/package/webaudio-node>
37. Client-Side WebAssembly Audio Effects (No Backend, Just the ... <https://dev.to/hexshift/client-side-webassembly-audio-effects-no-backend-just-the-browser-35f>
38. Best 10 Software Composition Analysis (SCA) Tools [2026] <https://www.ox.security/blog/software-composition-analysis-and-sca-tools/>
39. Top 10 Software Composition Analysis (SCA) Tools in 2026 <https://www.endorlabs.com/learn/best-sca-tools-05b7a>
40. 11 Best SCA Tools in 2026: Stop Supply Chain Attacks ... <https://www.plexicus.ai/blog/review/best-sca-tools-secure-software-supply-chain/>
41. Best SCA Tools for 2026: 9 Tools Compared <https://www.pixee.ai/blog/best-sca-tools-2026>
42. Top JavaScript PDF generator libraries for 2026 - Nutrient <https://www.nutrient.io/blog/top-js-pdf-libraries/>
43. Top JavaScript PDF Libraries in 2025: A Comprehensive ... <https://www.thatsoftwaredude.com/content/14087/top-javascript-pdf-libraries>
44. Comparing open source PDF libraries (2025 edition) <https://dev.to/joyfill/comparing-open-source-pdf-libraries-2025-edition-401g>
45. Building Large ZIP Files in the Browser Without Holding ... <https://medium.com/@certosinolab/building-large-zip-files-in-the-browser-without-holding-everything-in-memory-383037fa0e6a>
46. zip.js <https://bestofjs.org/projects/zipjs>
47. Open Source License Compatibility Checker - FOSSA <https://fossa.com/resources/license-compliance-tools/license-compatibility-checker/>
48. Open Source License Compatibility, Explained <https://www.mend.io/blog/license-compatibility/>
49. Software Supply Chain Security Beyond SBOMs: Sigstore, ... <https://aquilax.ai/blog/supply-chain-artifact-signing-slsa>

50. Verified Builds: SLSA Provenance and Sigstore Signing <https://guptadeepak.com/guides/verified-builds-slsa-sigstore/>
51. SLSA and Provenance: Your SBOM Says What's Inside, This ... <https://www.ostering.com/slsa-and-provenance/>
52. Software Supply Chain Security: SBOMs and Sigstore <https://www.matthewswong.com/en/blog/software-supply-chain-sbom-sigstore/>
53. How to sign an SBOM with Cosign <https://edu.chainguard.dev/open-source/sigstore/cosign/how-to-sign-an-sbom-with-cosign/>
54. SBOM, Signing & Attestation with GitHub Actions | Nine Lives <https://nineliveszerotrust.com/blog/container-sbom-signing-attestation/>
55. Supply Chain Security in CI: SBOMs, SLSA, and Sigstore <https://nathanberg.github.io/posts/supply-chain-security-ci-sbom-slsa-sigstore/>
56. Why HTML5-QRCode is the Leading JavaScript QR ... <https://scanapp.org/blog/2026/05/24/is-html5-qrcode-the-best-javascript-scanner-library.html>
57. QR Code generator library <https://github.com/nayuki/QR-Code-generator>
58. Client-side QR code generator with SVG output <https://news.ycombinator.com/item?id=41410442>
59. HyperFormula is an open-source headless spreadsheet ... <https://github.com/handsontable/hyperformula>
60. Best React Data Grids in 2026: AG ... <https://rv-grid.com/blog/top-5-react-datagrid-libraries-2026>
61. Best spreadsheet components for business applications <https://jspreadsheet.com/blog/best-spreadsheet-components-for-business>
62. Which rich text editor framework should you choose in 2025? <https://liveblocks.io/blog/which-rich-text-editor-framework-should-you-choose-in-2025>
63. Lexical <https://lexical.dev/>
64. The open-source rich text editor <https://tiptap.dev/product/editor>
65. Flow - Open-source ePub Reader <https://madewithreactjs.com/flow>
66. Thorium Reader <https://www.edrlab.org/software/thorium-reader/>
67. Ebook Reader <https://f-droid.org/en/categories/ebook-reader/>
68. SheetJS vs ExcelJS vs node-xlsx 2026: 20+ Formats vs ... <https://www.pkgpulse.com/guides/sheetjs-vs-exceljs-vs-node-xlsx-excel-files-node-2026>
69. SheetJS vs ExcelJS vs hucre vs excelize-wasm vs xlsx-populate <https://jstool.gitlab.io/blog/posts/javascript-excel-xlsx-libraries-comparison/>
70. I Tested Three Node.js Excel Libraries So You Don't Have To <https://mfyz.com/nodejs-excel-library-comparison/>

71. The Best ffmpeg.wasm Alternatives for Client-Side Video ... <https://dayverse.id/en/articles/best-ffmpeg-wasm-alternatives-client-side/>
72. Overview - FFmpeg.wasm - Netlify <https://ffmpegwasm.netlify.app/docs/overview/>
73. WebAssembly for Web Developers: Getting Started with ... <https://daily.dev/blog/webassembly-getting-started-wasm-web-developers/>
74. FFmpeg + WebAssembly <https://dev.to/alfg/ffmpeg-webassembly-2cbl>
75. App State Management: Architecture and Sync Guide <https://capgo.app/blog/app-state-management/>
76. Offline-First Apps: Architecture, Frameworks & Real ... <https://www.locize.com/blog/offline-first-apps>
77. Everything you need to know to create offline-first web apps. <https://github.com/pazguille/offline-first>
78. ChristopherVR/pptx-viewer <https://github.com/ChristopherVR/pptx-viewer/>
79. Ask HN: What alternatives to Powerpoint/Prezi are there? <https://news.ycombinator.com/item?id=15575363>
80. Home | PptxGenJS <https://gitbrent.github.io/PptxGenJS/>
81. xuri/excelize-wasm <https://github.com/xuri/excelize-wasm>
82. Client-Side Excel File Rendering: No Server Round-Trip ... <https://apryse.com/capabilities/spreadsheet-editor/client-side-spreadsheet-processing>
83. Creating the Local First Stack - Ersin's Blog <https://www.ersin.nz/articles/creating-the-local-first-stack>
84. SQLite WASM + OPFS: Local-First Web Apps With Real SQL <https://kvassiliou.com/tech/sqlite-wasm-opfs-local-first-web-apps-2026>
85. Run a Full SQLite Database in the Browser (Wasm + OPFS) <https://www.instagram.com/reel/DWOubApgpdV/>
86. The browser is your database: Local-first comes of age <https://www.infoworld.com/article/4133648/the-browser-is-your-database-local-first-comes-of-age.html>
87. How Local-First Development Is Changing How We Make ... <https://www.heavybit.com/library/article/local-first-development>
88. LocalStorage vs. IndexedDB vs. Cookies vs. OPFS ... <https://rxdb.info/articles/localstorage-indexeddb-cookies-opfs-sqlite-wasm.html>
89. Leader election in browser tabs, the easy way - Green Vitriol <https://greenvitriol.com/posts/browser-leader>
90. PGLite vs Electric SQL vs Triplit for Local-First Apps (2026) <https://www.pkgpulse.com/guides/pglite-vs-electric-sql-vs-triplit-local-first-databases-2026>

91. Comparison of Web Worker, SharedWorker, and Service ... <https://dev.to/zeeshanali0704/comparison-of-web-worker-sharedworker-and-service-worke-3ff0>
92. SharedWorker: Browser Support, API, Limitations <https://www.testmuai.com/learning-hub/sharedworker-browser-support/>
93. paulmillr/noble-hashes: Audited & minimal JS ... <https://github.com/paulmillr/noble-hashes>
94. Noble cryptography - Paul Miller <https://paulmillr.com/noble/>
95. Web Crypto API - MDN Web Docs https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API
96. Exploring SHA-256 Performance on the Browser ... <https://medium.com/@ronantech/exploring-sha-256-performance-on-the-browser-browser-apis-javascript-libraries-wasm-webgpu-9d9e8e681c81>
97. 101arrowz/fflate: High performance (de)compression ... <https://github.com/101arrowz/fflate>
98. fflate vs pako vs Node.js zlib 2026 <https://www.pkgpulse.com/guides/fflate-vs-pako-vs-node-zlib-compression-javascript-2026>
99. Wasm compression benchmarks and the cost of missing ... <https://nickb.dev/blog/wasm-compression-benchmarks-and-the-cost-of-missing-compression-apis/>
100. ruilisi/fortune-sheet: A drop-in javascript spreadsheet ... <https://github.com/ruilisi/fortune-sheet>
101. FortuneSheet - Best JavaScript Data Grids and Spreadsheets ... <https://jspreadsheets.com/fortune-sheet/>
102. Univer vs. Luckysheet: A Comprehensive Comparison of ... <https://blog.univer.ai/posts/univer-vs-luckysheet-a-comprehensive-comparison-of-open-source-spreadsheet-solutions/>
103. GitHub - dream-num/univer: The Office Harness for AI Agents <https://github.com/dream-num/univer>
104. SVG-Edit/svgedit: Powerful SVG-Editor for your browser <https://github.com/svg-edit/svgedit>
105. EditSvgCode — An AI-Powered Online SVG Editor <https://unmanagedvisio.com/editsvgcode-an-ai-powered-online-svg-editor/>
106. Konva - JavaScript Canvas 2d Library <https://konvajs.org/>
107. Paper.js — About <https://paperjs.org/about/>
108. Best JavaScript Canvas Library in 2026 — How to Choose <https://konvajs.org/docs/guides/best-canvas-library.html>

109. Konva.js vs Fabric.js: In-Depth Technical Comparison and ... <https://medium.com/@www.blog4j.com/konva-js-vs-fabric-js-in-depth-technical-comparison-and-use-case-analysis-9c247968dd0f>
110. Best Canvas Libraries for Web & Mobile (Jan 2026) - Velt <https://velt.dev/blog/best-canvas-library-web-mobile-apps>
111. konva/CHANGELOG.md at master <https://github.com/konvajs/konva/blob/master/CHANGELOG.md>
112. silvia-odwyer/photon: Rust/WebAssembly image ... <https://github.com/silvia-odwyer/photon>
113. ffmpeg.wasm | ffmpeg.wasm <https://ffmpegwasm.netlify.app/>